

Chapter 9

Main Memory

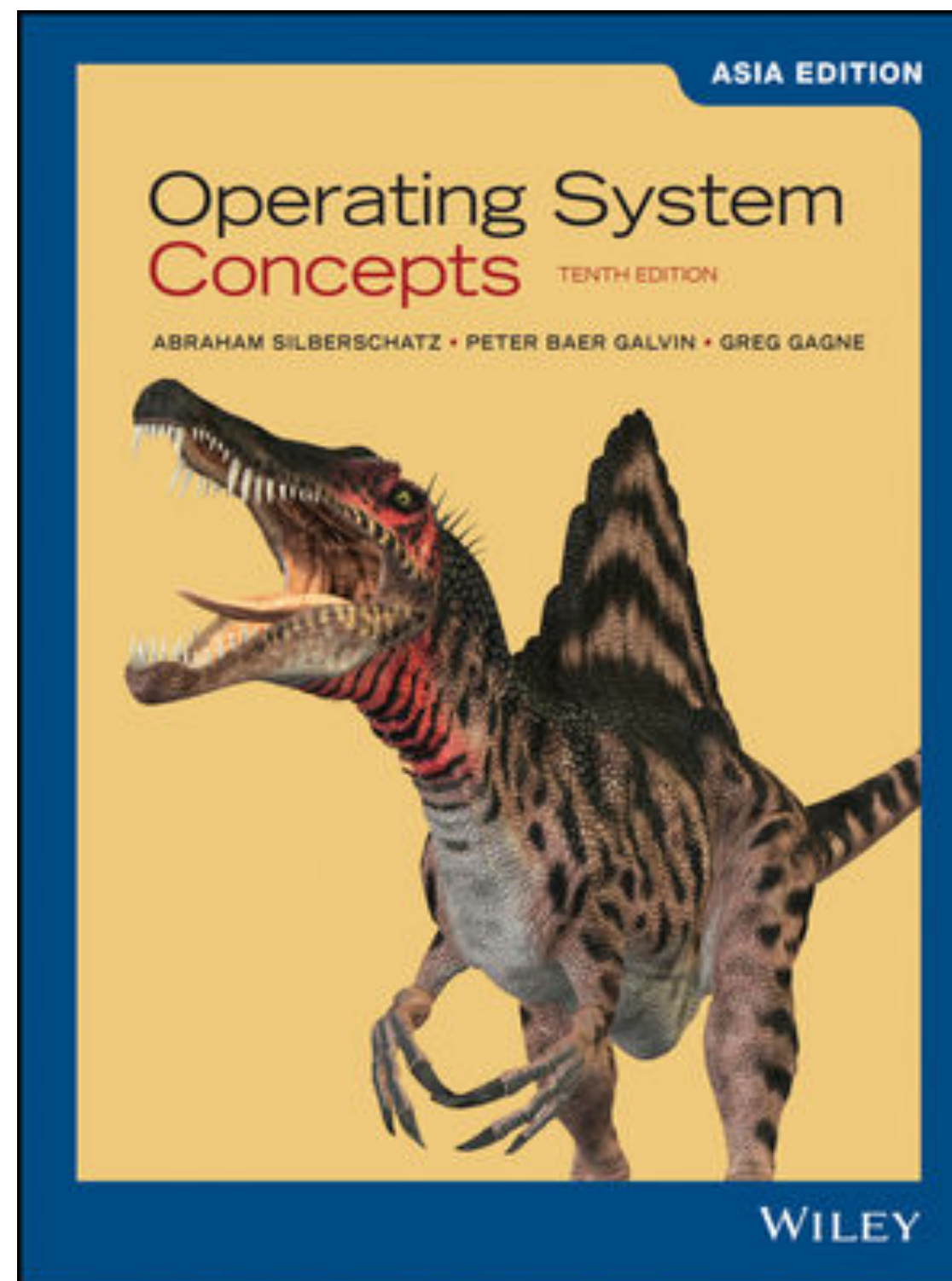
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/7/31

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping

Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping

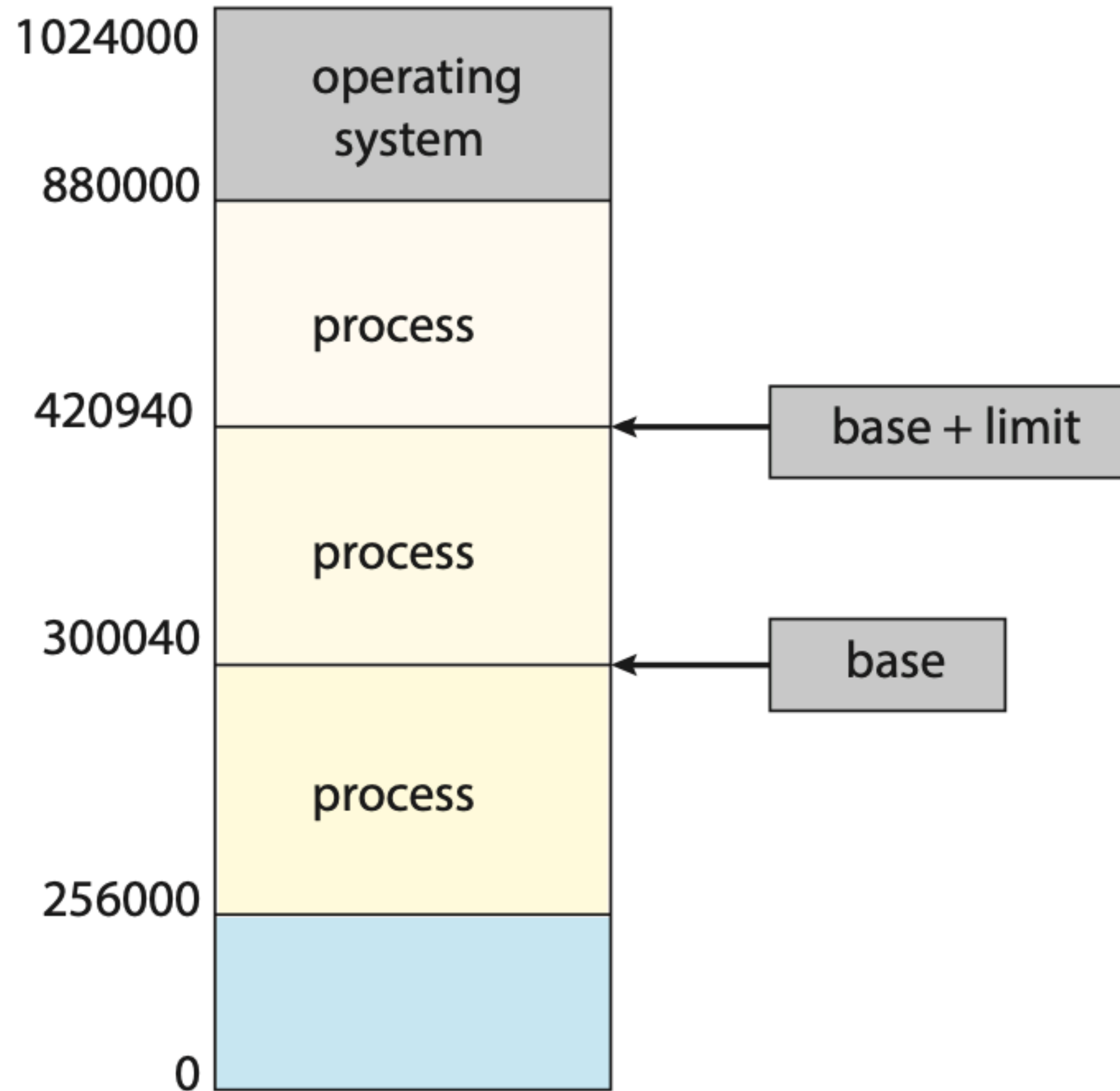
Memory

- Contents
 - Program Codes
 - The CPU fetches instructions from memory according to the value of the program counter
 - Data
 - These instructions may cause additional loading from and storing to specific memory addresses
 - data / heap /stack

Subsections

- (9.1.1) Basic Hardware
- (9.1.2) Address Binding 位址繫結
- (9.1.3) Logical Versus Physical Address Space
- (9.1.4) Dynamic Loading
- (9.1.5) Dynamic Linking and Shared Libraries

Protection of Memory Space



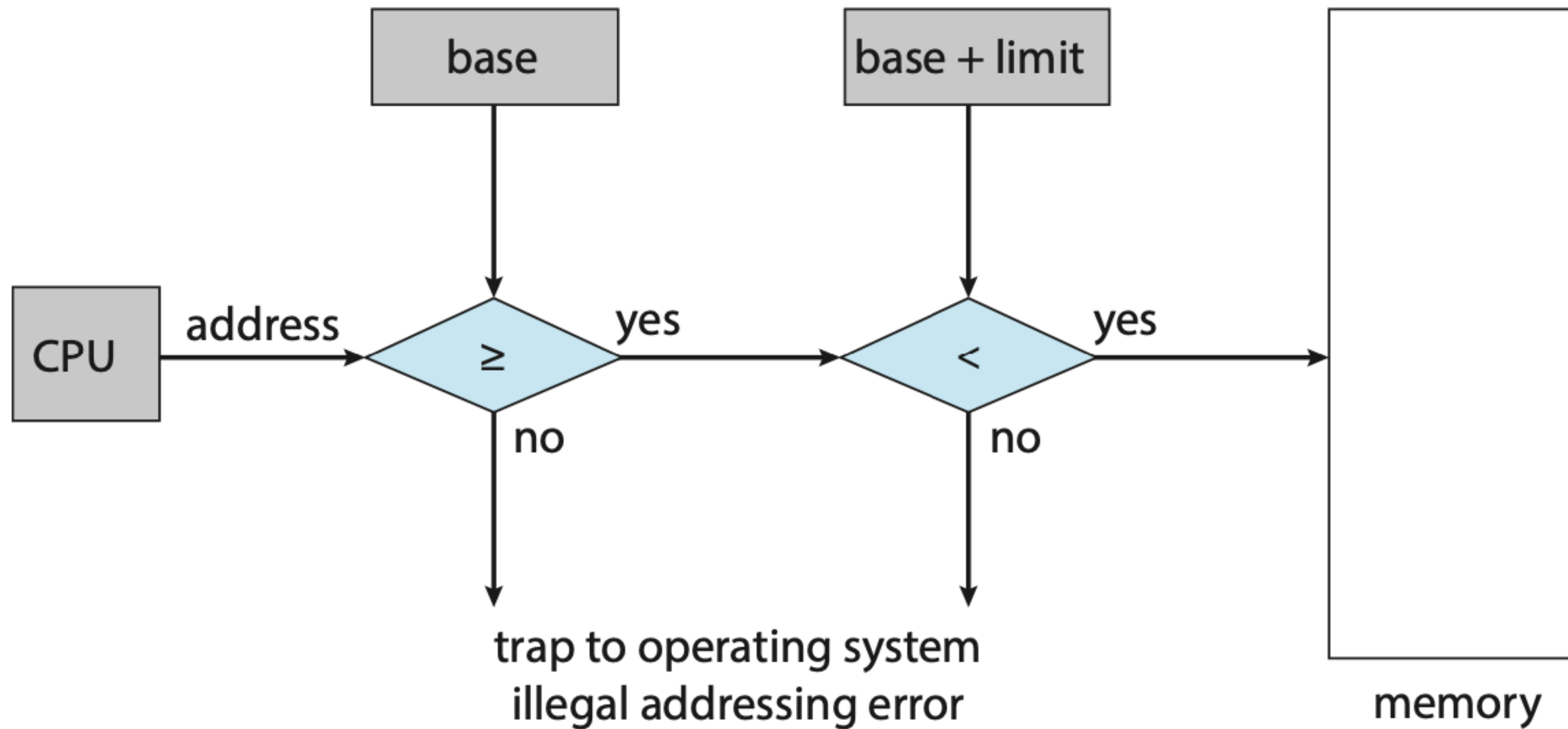
Example:

base: 300040

limit: 120900

The program can legally access all addresses from 300040 through 420939 (inclusive)

Protection of Memory Space



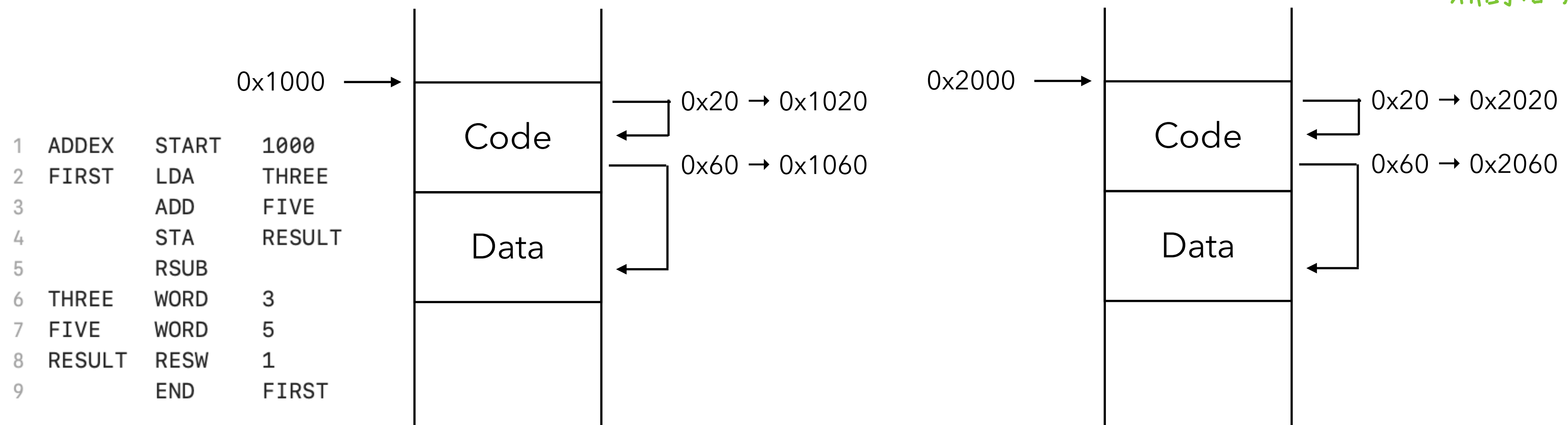
Subsections

- (9.1.1) Basic Hardware
- (9.1.2) Address Binding
- (9.1.3) Logical Versus Physical Address Space
- (9.1.4) Dynamic Loading
- (9.1.5) Dynamic Linking and Shared Libraries

Address Binding

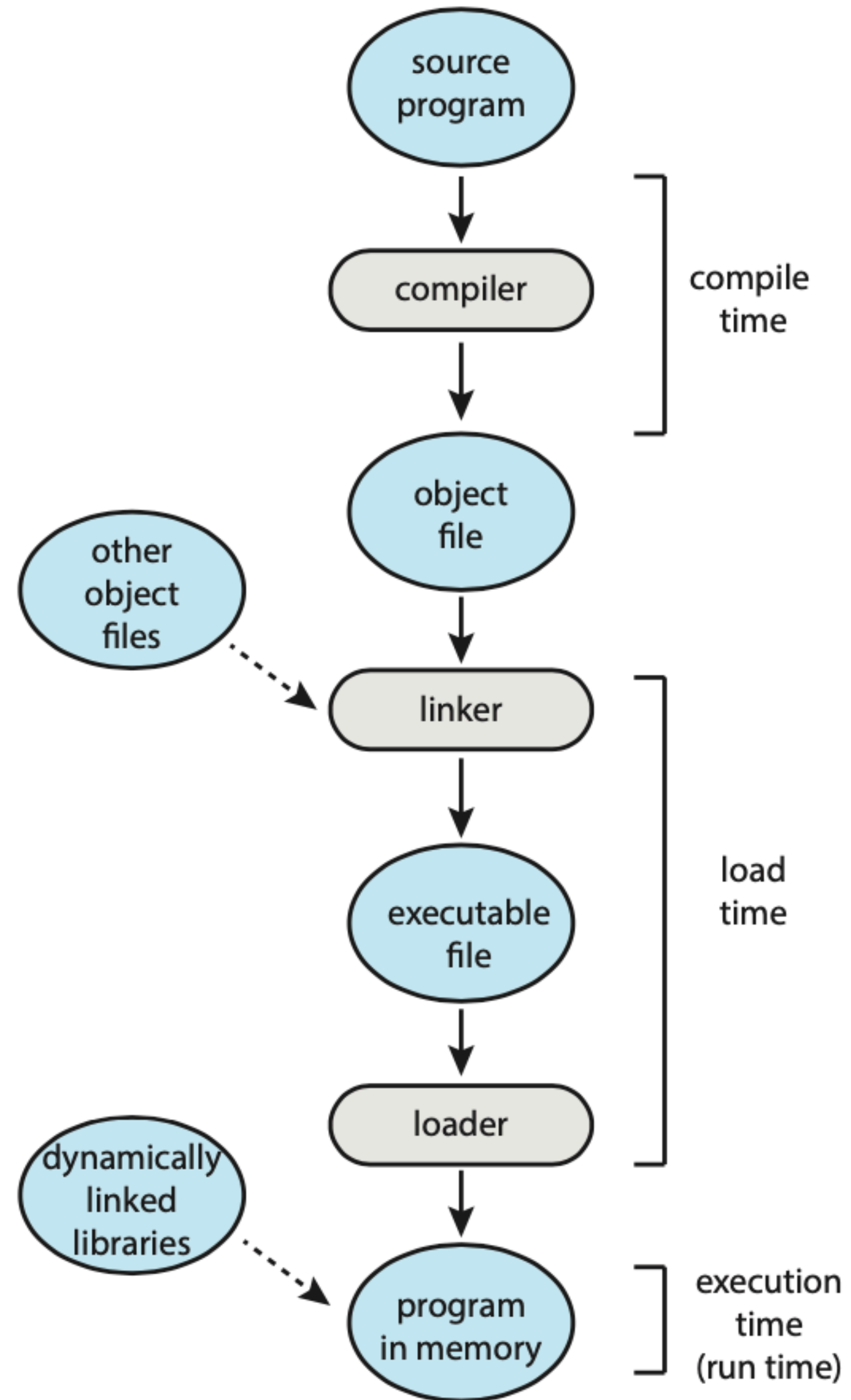
位址繫結

- The main purpose of address binding is to map symbolic or logical addresses in a program to actual memory addresses so that the program can be correctly loaded into memory and executed.
- Most systems allow a user process to reside in any part of the physical memory. (relocatable code)
- A compiler/assembler typically binds the symbolic addresses to relocatable addresses (可被重新載入的位址) (想像成一個相對的概念)
- The linker/loader in turn binds the relocatable addresses to absolute addresses (每個程式都會被重分配到一個新的相對位址)



Address Binding

- The binding of instructions and data to memory addresses can be done at any step
 - Compile time
 - If you know at compile time where the process will reside in memory, then **absolute code** can be generated
 - Boot → 開機程式的 Code 通常都是 absolute 的
 - Load time
 - If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code**
 - Execution time
 - If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time
 - **Paging** is a memory management mechanism that implements execution-time address binding.

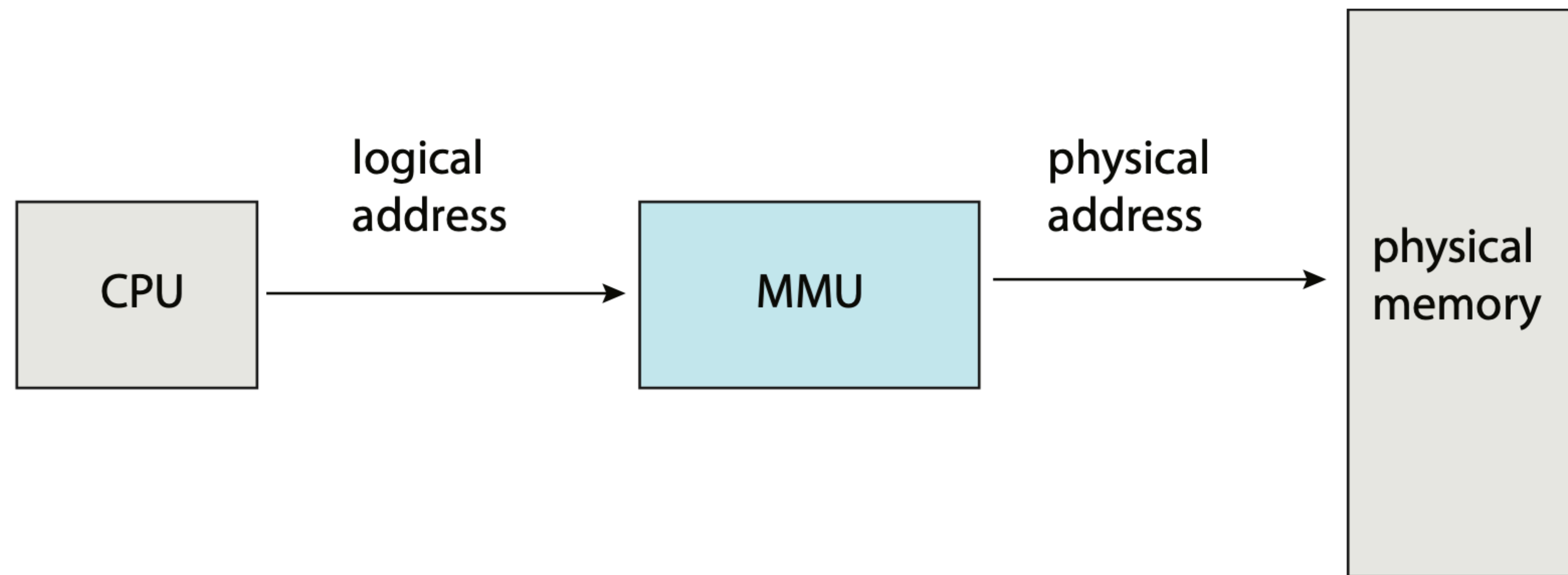


Subsections

- (9.1.1) Basic Hardware
- (9.1.2) Address Binding
- (9.1.3) Logical Versus Physical Address Space
- (9.1.4) Dynamic Loading
- (9.1.5) Dynamic Linking and Shared Libraries

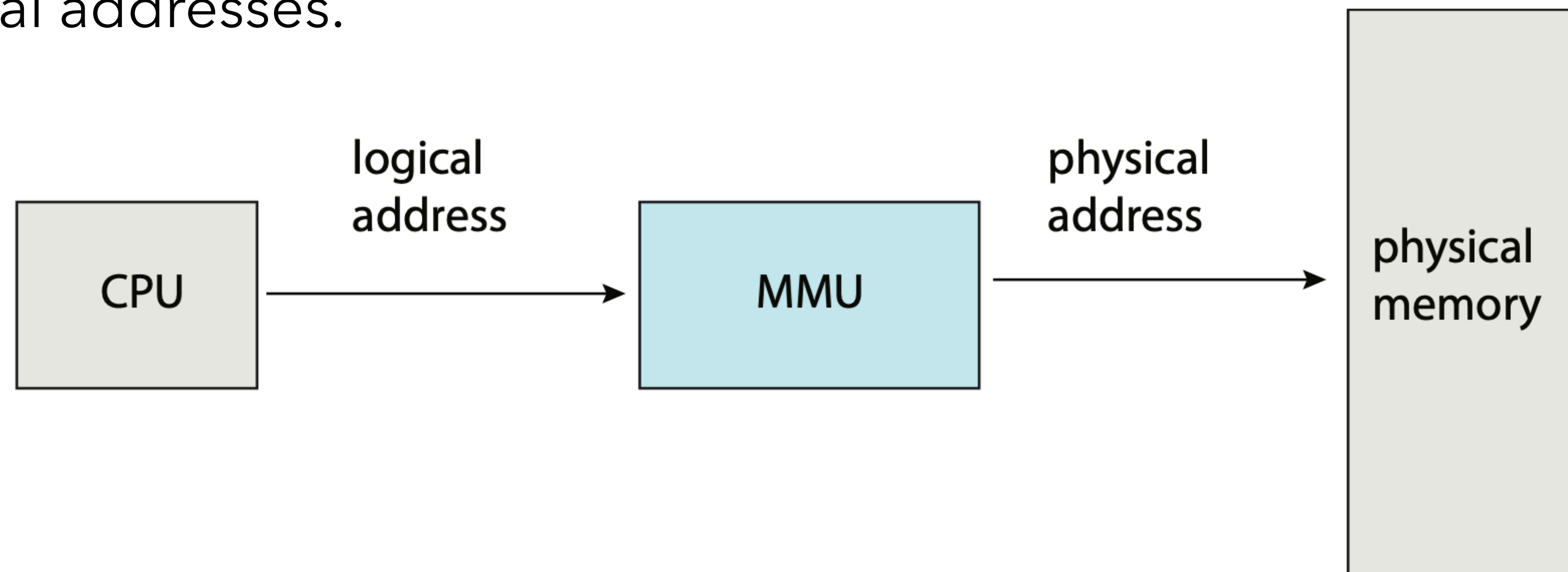
Logical Versus Physical Address Space

- An address generated by the CPU is referred to as a **logical address** (**virtual address**)
- An address seen by the memory unit is referred to as a **physical address**



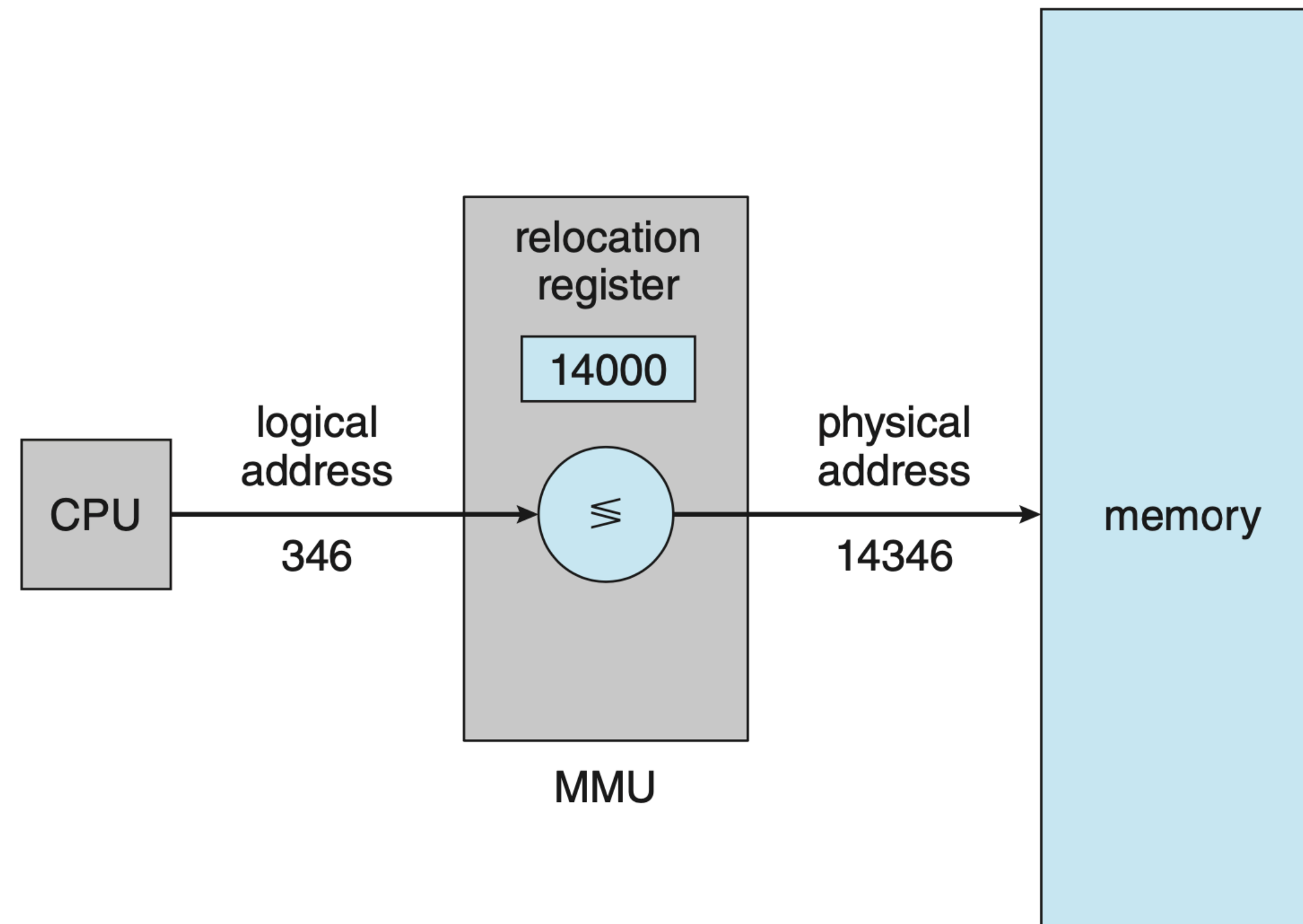
Logical Versus Physical Address Space

- Binding addresses at either **compile** or **load** time generates identical logical and physical addresses
 - When a loader loads codes into memory, the loader will modify the memory accordingly. Thus, when a CPU fetches an instruction, the addresses are physical addresses.
- However, the **execution-time** address-binding scheme results in differing logical and physical addresses.



Logical Versus Physical Address Space

- The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (MMU)
- We can choose from many different methods to accomplish such mapping as we will discuss later
- A simple example



Subsections

- (9.1.1) Basic Hardware
- (9.1.2) Address Binding
- (9.1.3) Logical Versus Physical Address Space
- (9.1.4) Dynamic Loading
- (9.1.5) Dynamic Linking and Shared Libraries

Dynamic Loading

- So far, it has been necessary for the entire program and all data of a process to be in physical memory for the process to execute.
- The size of a process has thus been limited to the size of physical memory.
- To obtain better memory-space utilization, we can use **dynamic loading**.
- With dynamic loading, a routine is not loaded until it is called.

Subsections

- (9.1.1) Basic Hardware
- (9.1.2) Address Binding
- (9.1.3) Logical Versus Physical Address Space
- (9.1.4) Dynamic Loading
- (9.1.5) Dynamic Linking and Shared Libraries

Dynamic Linking and Shared Libraries

共享函式庫

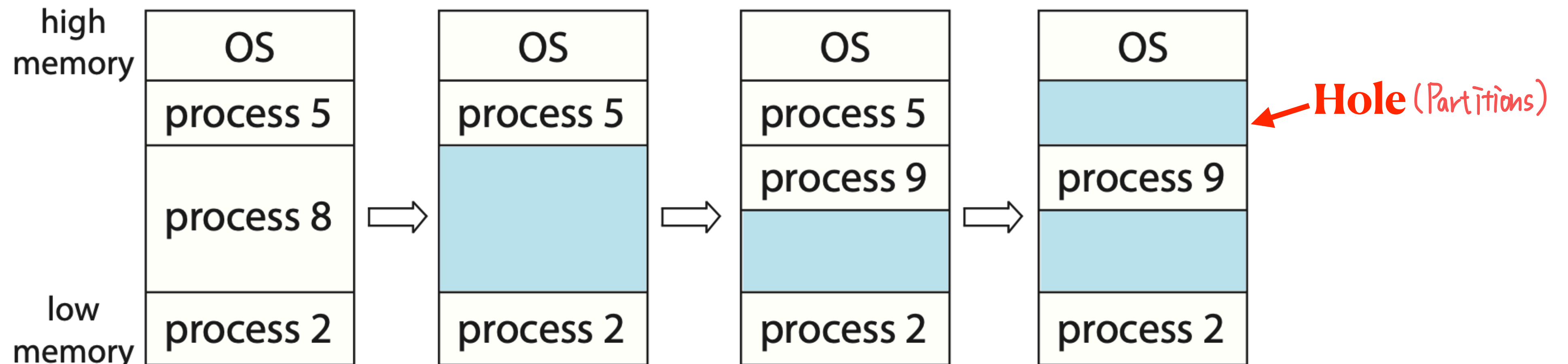
- Static Linking
 - System libraries are treated like any other object module and are combined by the loader into the binary program image.
- Dynamically linked libraries (DLLs) are system libraries that are linked to user programs when the programs are run.
 - .so / .dll

Outline

- Background
- Contiguous Memory Allocation → 將Process 配置於連續記憶體空間 → 有效能問題!
- Paging
- Structure of the Page Table
- Swapping

Contiguous Memory Allocation

- Early method 早期的做法 (会有效能問題)
- In contiguous memory allocation, each process is contained in a **single section** of memory that is **contiguous** to the section containing the next process.
- Assign processes to **variably sized** partitions in memory, where each partition contain exactly one process.
- The **OS** keeps a table indicating which parts of memory are available and which are occupied.



Contiguous Memory Allocation

常考!

當一個記憶體空間中有多个 Partition (Assume 沒有外部斷裂), 該選哪個放?

- When there are several holes in memory, how to select a free hole for a new coming process?
- The **first-fit**, **best-fit**, and **worst-fit** strategies are the ones most commonly used to select a free hole from the set of available holes.
 - **First-fit**: Allocate the first hole that is big enough. 找到的第一个能放的, 就放那
 - **Best-fit**: Allocate the smallest hole that is big enough. This strategy produces the smallest leftover hole. 找最符合大小的(浪费最少空間的)
 - **Worst-fit**: Allocate the largest hole. This strategy produces the largest leftover hole, which may be **more useful** than the smaller leftover hole.

└ 找「浪费最多」的

② Why: Best Fit 的話, 剩下浪费的空間可能就真的浪费了, 但 Worst Fit 的話, 剩下浪费的空間或许可以給其他 Process 來用

Example

- Given five memory partitions of 120 Kb, 500 Kb, 230 Kb, 320 Kb, 600 Kb (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of 230 Kb, 420 Kb, 150 Kb, and 430 Kb (in order)?
 - First-fit: *找到第一个能放的*
 - 230 Kb is put in 500 Kb partition (120 Kb, *记得要500-230 算剩下的 Partition* 270 Kb, 230 Kb, 320 Kb, 600 Kb)
 - 420 Kb is put in 600 Kb Partition (120 Kb, 270 Kb, 230 Kb, 320 Kb, 180 Kb)
 - 150 Kb is put in 270 Kb Partition (120 Kb, 120 Kb, 230 Kb, 320 Kb, 180Kb)
 - 430 Kb must wait (*发生 External fragmentation*)
 - (Homework) Best-fit, Worst-fit

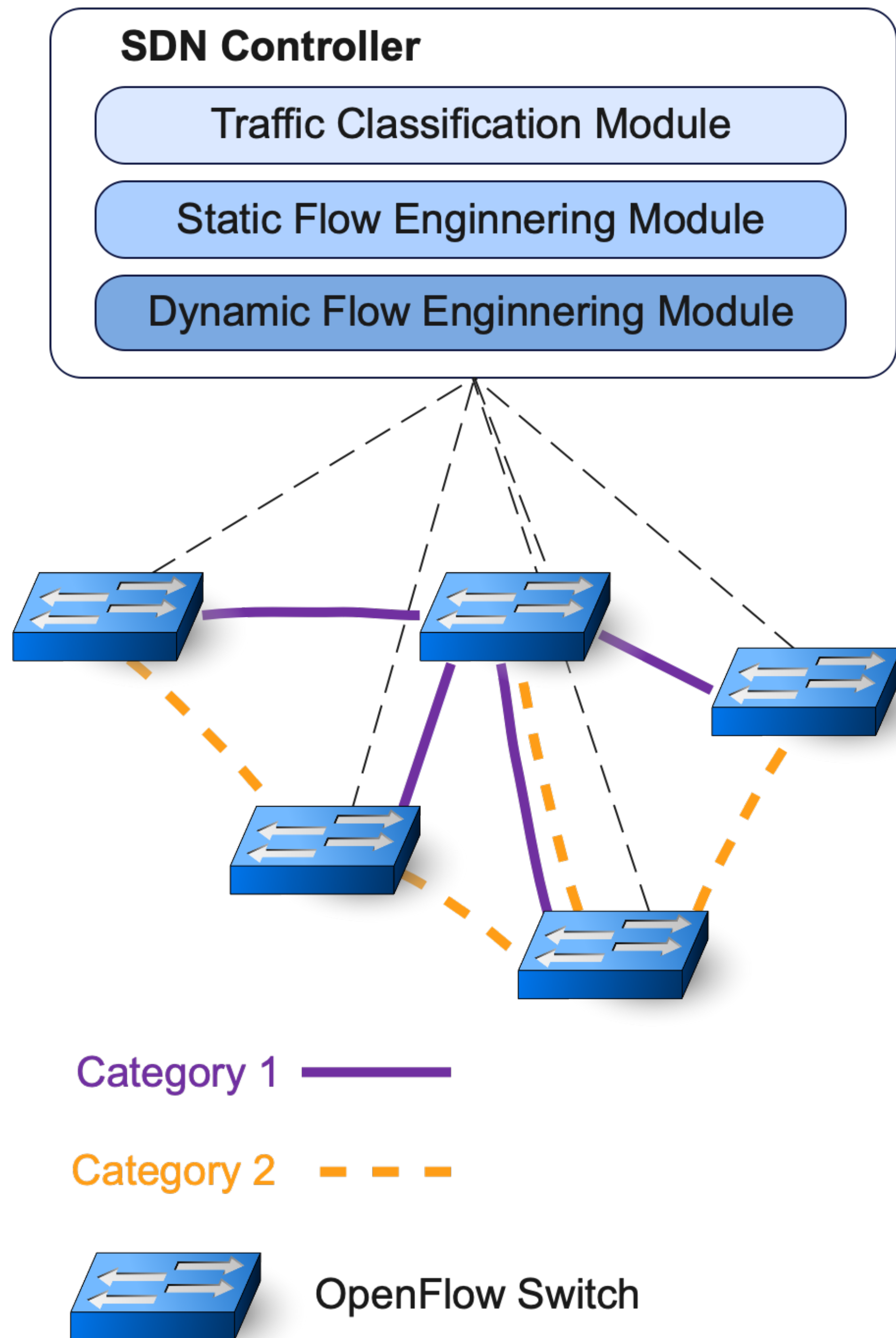
Example

- Given five memory partitions of 120 Kb, 500 Kb, 230 Kb, 320 Kb, 600 Kb (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of 230 Kb, 420 Kb, 150 Kb, and 430 Kb (in order)?
- Best-fit: (找「浪费最少」记忆体空間的)
 - 230 Kb is put in 230 Kb partition (120 Kb, 500 Kb, 0 Kb, 320 Kb, 600 Kb)
 - 420 Kb is put in 500 Kb Partition (120 Kb, 80 Kb, 0 Kb, 320 Kb, 600 Kb)
 - 150 Kb is put in 320 Kb Partition (120 Kb, 80 Kb, 0 Kb, 170 Kb, 600Kb)
 - 430 Kb is put in 600 Kb Partition (120 Kb, 80Kb, 0 Kb, 170Kb, 170 Kb)

Example

- Given five memory partitions of 120 Kb, 500 Kb, 230 Kb, 320 Kb, 600 Kb (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of 230 Kb, 420 Kb, 150 Kb, and 430 Kb (in order)?
 - Worst-fit: 找浪费最多的
 - 230 Kb is put in 600 Kb partition (120 Kb, 500 Kb, 230 Kb, 320 Kb, 370 Kb)
 - 420 Kb is put in 500 Kb Partition (120 Kb, 80 Kb, 230 Kb, 320 Kb, 370 Kb)
 - 150 Kb is put in 370 Kb Partition (120 Kb, 80 Kb, 230 Kb, 320 Kb, 220 Kb)
 - 430 Kb must wait

```
int main(int argc, const char * argv[]) {  
  
    // TextBook  
    int partitions[PARTITIONS] = {120, 500, 230, 320, 600};  
    int processes[PROCESSES] = {230, 420, 150, 430};  
  
    // cout << "First Fit:\n";  
    // firstFit(partitions, processes);  
  
    // cout << "Best Fit:\n";  
    // bestFit(partitions, processes);  
  
    cout << "Worst Fit:\n";  
    worstFit(partitions, processes);  
  
    return 0;  
}
```



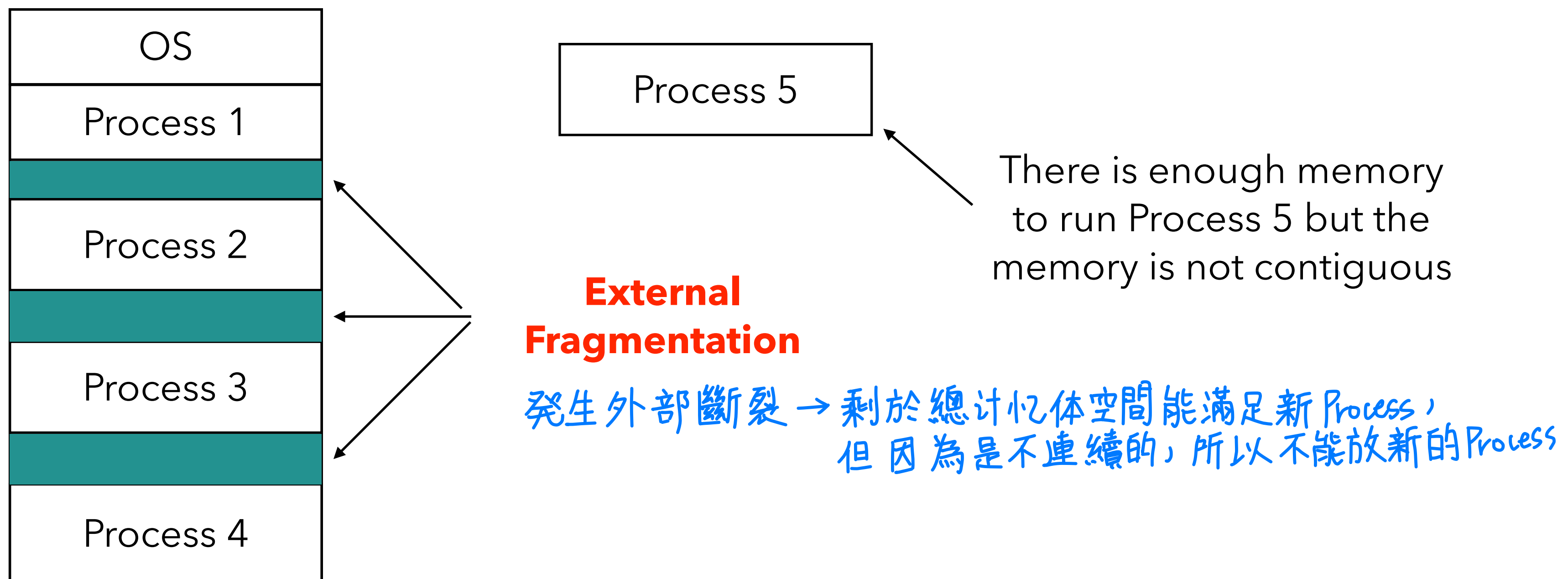
- **RANDOM:** This approach will select one path from P' randomly.
- **MIN:** This approach will select a path with the smallest available bandwidth (i.e., $\min_{p \in P'} \{p.availb\}$) in P' .
- **MAX:** This approach will select a path with the largest available bandwidth (i.e., $\max_{p \in P'} \{p.availb\}$) in P' .

External Fragmentation

会考!

外部斷裂

- External fragmentation
 - External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous
 - One solution is **compaction**. (Move all processes toward one end of memory.)



Another Problem

- Consider a multiple-partition allocation scheme with a hole of 18,464 bytes
- Suppose that the next process requests 18,462 bytes
- We are left with a hole of 2 bytes
- The overhead to keep track of this hole is larger than the hole itself

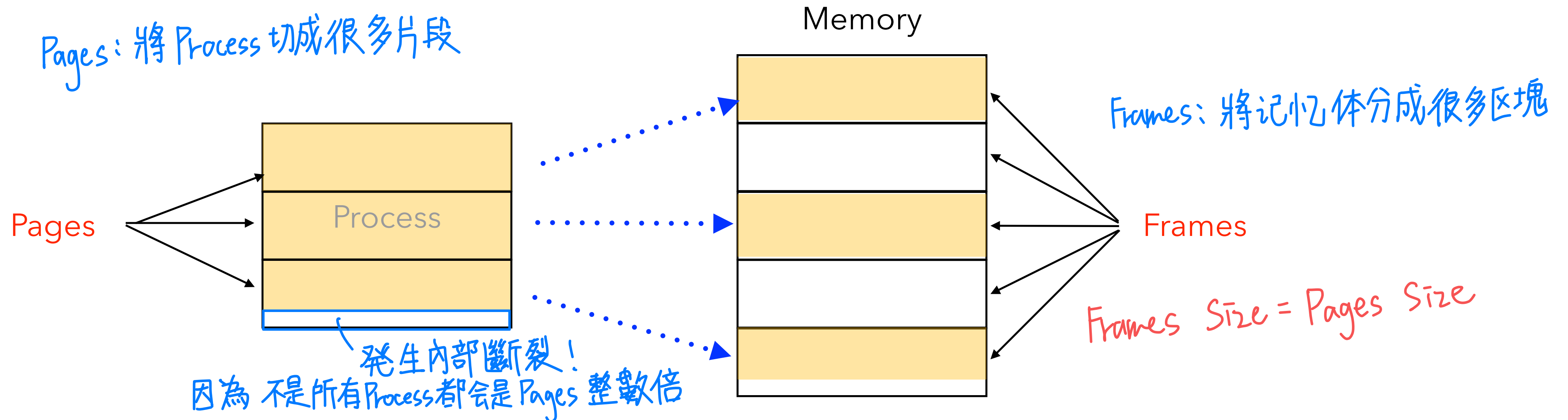
Outline

- Background
- Contiguous Memory Allocation
- **Paging**
- Structure of the Page Table
- Swapping

Paging

現在依然在使用的技術
e.g. linux 中一個 Pages 4kb

- A memory-management scheme that permits a process's physical address space to be noncontiguous
- Paging avoids external fragmentation and the associated need for compaction
- Because it offers numerous advantages, paging is used in most OSs
- Break physical memory into fixed-sized blocks called frames and break logical memory into blocks of the same size called pages

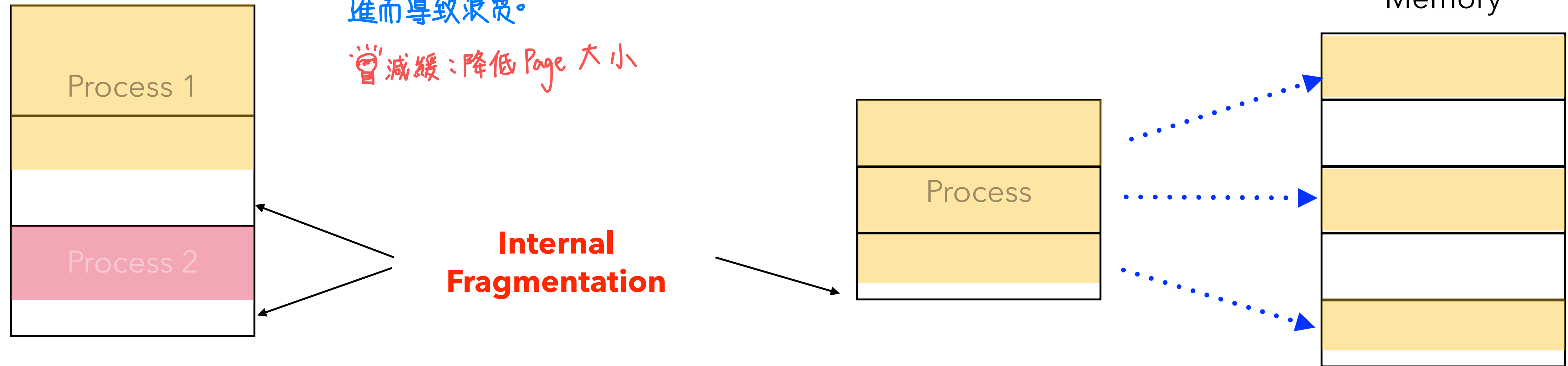


Internal Fragmentation

內部斷裂

不是所有 Process 都是 Pages 的整數倍，為了補足 Pages Size，會讓 Process 大小補到 Pages 整數倍，進而導致浪費。

🧠減緩：降低 Page 大小



We expect internal fragmentation to average one-half page per process.

$$\frac{1}{2}(\text{PageSize})$$

Basic Operation Method

為每個 Process 都建立一個 Page Table
來轉換到 Physical address

Functions of the Page Table in Paging:

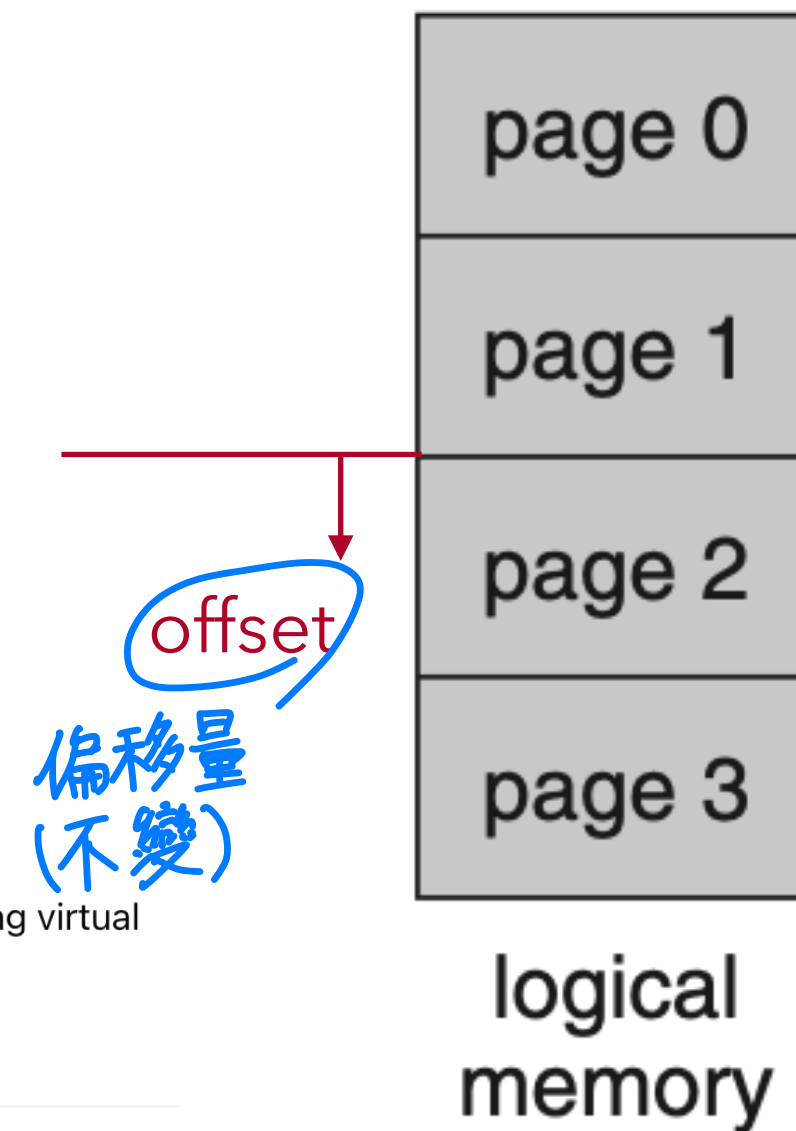
In a paging-based memory management system, the **page table** plays a crucial role in translating virtual addresses into physical addresses and managing memory access. Its key functions include:

1. Address Translation

- The primary role of the page table is to map **virtual page numbers (VPN)** to **physical frame numbers (PFN)**.
- When a process generates a virtual address, the Memory Management Unit (MMU) uses the page table to translate it into a physical address.

2. Memory Protection

- Page tables include **protection bits** (e.g., read, write, execute) to control access rights to each page.
- This prevents processes from accessing memory illegally and supports isolation between processes.



offset
偏移量
(不變)

logical addresses
(0~2^m)

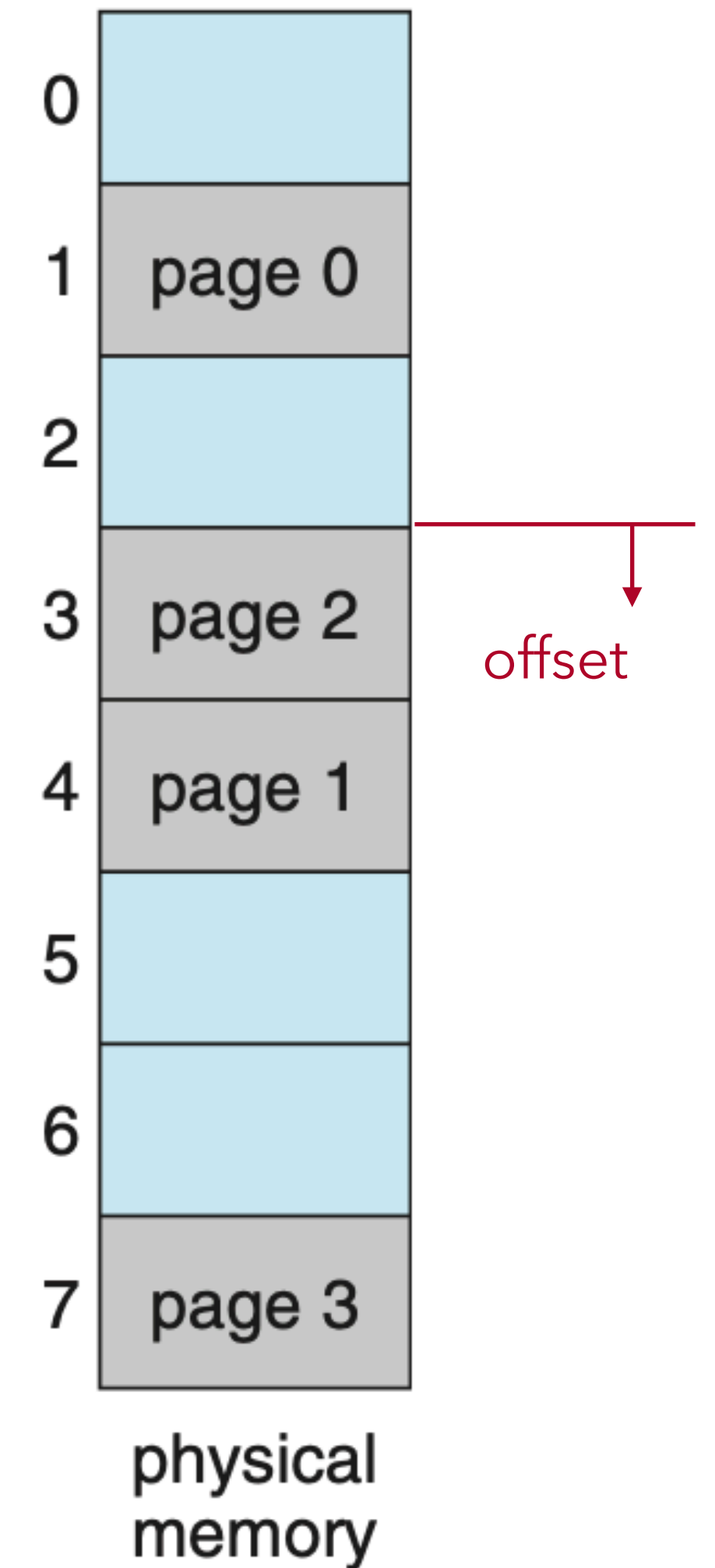
原始 Process (邏輯mem)

0	1
1	4
2	3
3	7

page table

Per-process
Page Table

frame
number

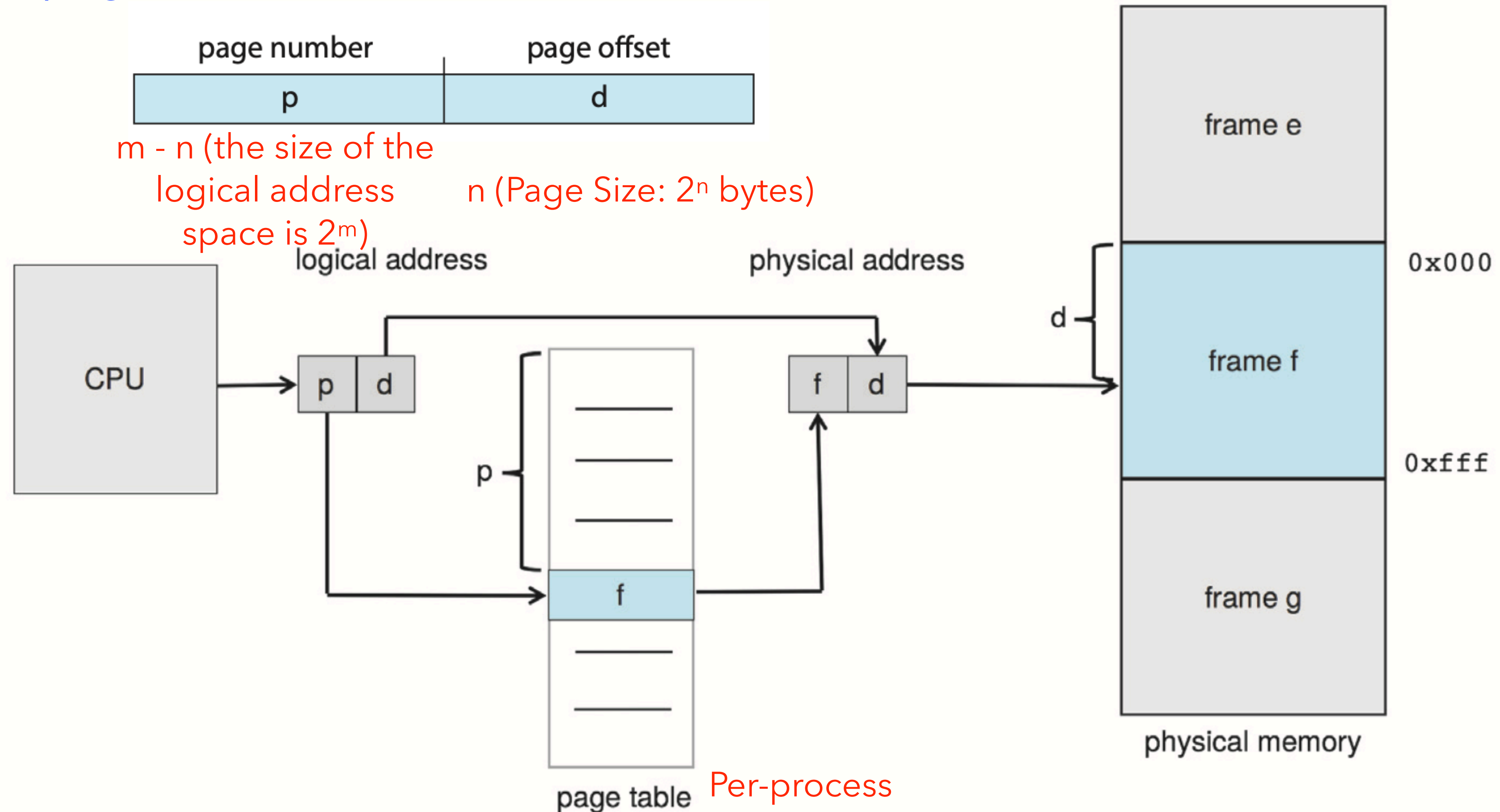


物理mem
空間



Basic Operation Method

- Every address generated by the CPU is divided into two parts: a **page number** (p) and a **page offset** (d)



Example (n = 2, m = 4)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Logical address 5 (page 1, offset 1)
maps to physical address 25 ($6 \times 4 + 1$)

Logical address 14 (page 3, offset 2)
maps to physical address 10 ($2 \times 4 + 2$)

ChatGPT

我是作業系統的老師, 幫我出一題 paging 系統中, logical address 轉換成 physical address 的計算題

不考!

當然可以，以下是一題適合給作業系統課程學生的 **Paging 系統位址轉換計算題**，附有詳細設定與計算目標：

 【題目】 Logical Address to Physical Address Translation

Assume a **paging system** with the following characteristics:

- The system uses **32-bit logical addresses**.
- The **page size** is **4 KB** (i.e., 2^{12} bytes).
- The system has a **single-level page table**.
- The **page table** for a given process has the following mappings:

Page Number	Frame Number
0x00000	0x00005
0x00001	↓ 0x0002A

ChatGPT

Assume a **paging system** with the following characteristics:

- The system uses **32-bit logical addresses**.
- The **page size** is **4 KB** (i.e., 2^{12} bytes).
- The system has a **single-level page table**.
- The **page table** for a given process has the following mappings:

Page Number	Frame Number
0x00000	0x00005
0x00001	0x0002A
0x00002	0x0001F
0x00003	0x0003C

Question:

Given a logical address of `0x00012FAB` , calculate the **corresponding physical address**. Show your steps.

ChatGPT

✅【參考解法提示給老師使用】（請隱藏或自行調整）

1. Page size = 4 KB = 2^{12} bytes, so:

- Offset = lower 12 bits
- Page number = upper 20 bits

2. Step 1: Convert 0x00012FAB to binary:

0000 0000 0000 0001 0010 1111 1010 1011

3. Step 2: Extract:

- Page number = upper 20 bits = 0x00012 = page 0x00012
- Offset = lower 12 bits = 0xFAB

4. Step 3: Is page 0x00012 in the table?

- ❌ No → Page Fault

5. ✅ 若要讓學生能算出結果，可改題目地址為：0x00001FAB

- Page number = 0x00001, Offset = 0xFAB
- Frame number = 0x0002A

6. Physical address = (Frame number × page size) + offset

= 0x0002A × 0x1000 + 0xFAB

= 0x2A000 + 0xFAB = 0x2AFAB



Page Size

Page size 太大 → Internal Fragmentation 影響嚴重

- Too large? Internal Fragmentation

- Too Small? Overhead to maintain page/frame tables.

Page size 太小 → 增加 Page Table 管理負擔

- Generally, page sizes have grown over time as processes, data sets, and main memory have become larger.
- Today, pages are typically either 4 KB or 8 KB in size

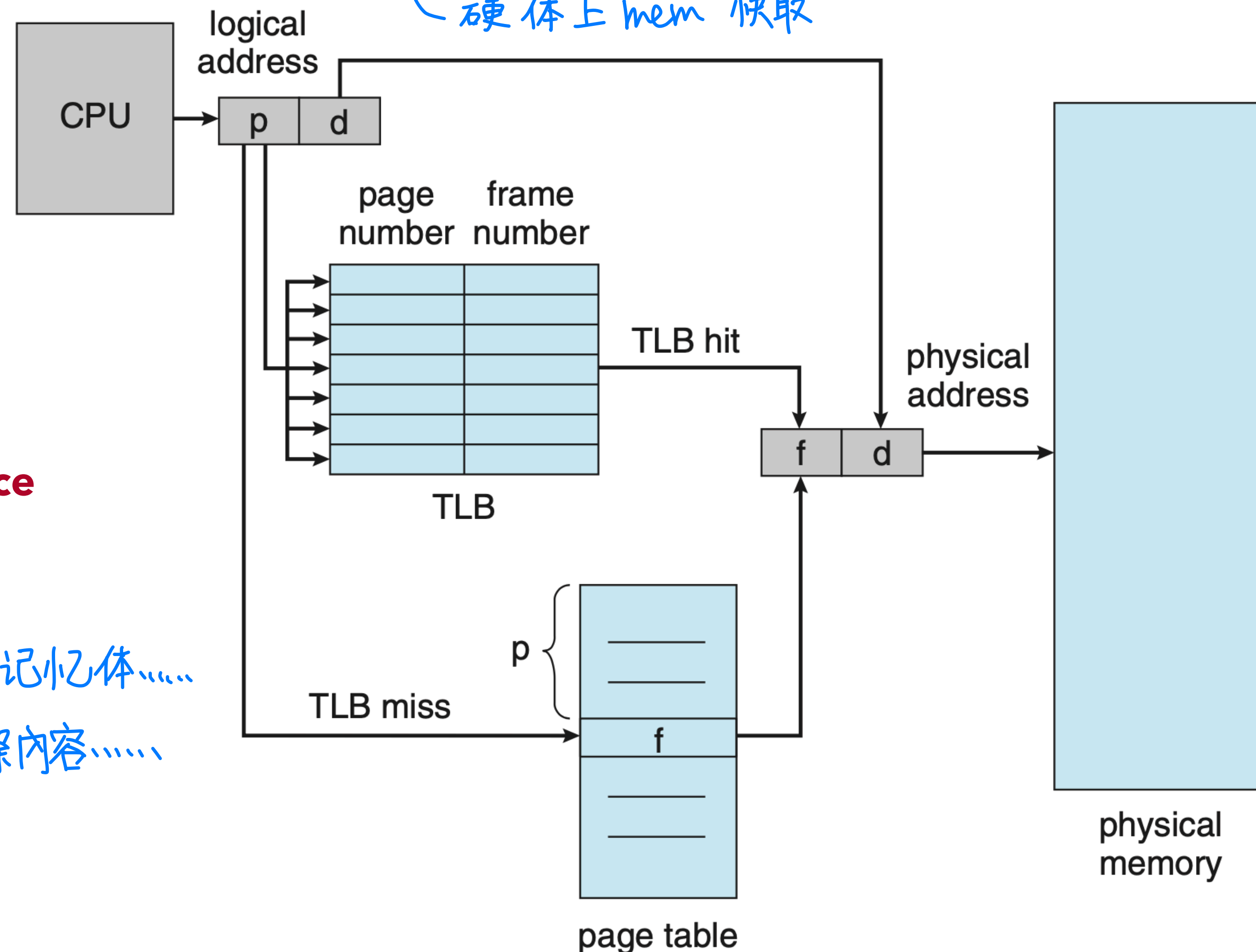
Hardware Support

The TLB (translation look-aside buffer) contains only a few of the page-table entries. (Which entries?)

└ 硬体上 mem 快取

Access memory twice

有了Pages, 可能要找两次记忆体.....
先查 Page Table, 再找实际内容.....
→ 硬体加速!



Protection

📖 Functions of the Page Table in Paging:

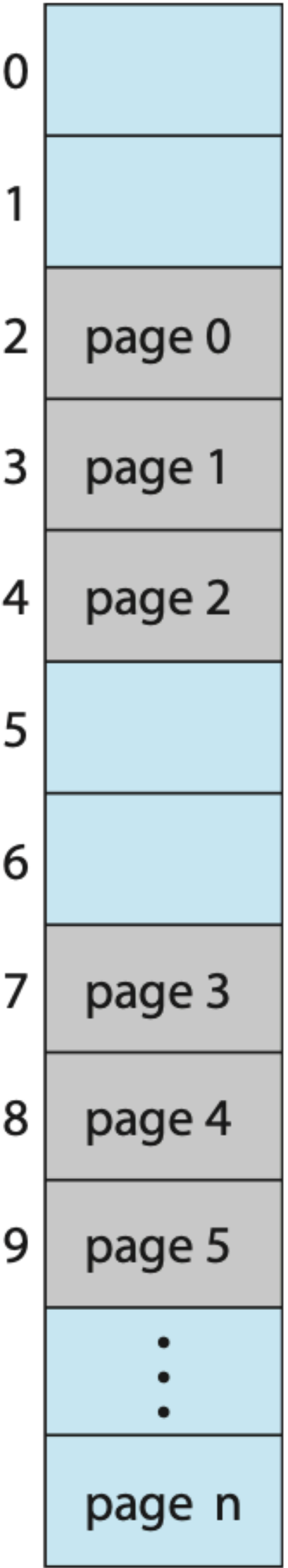
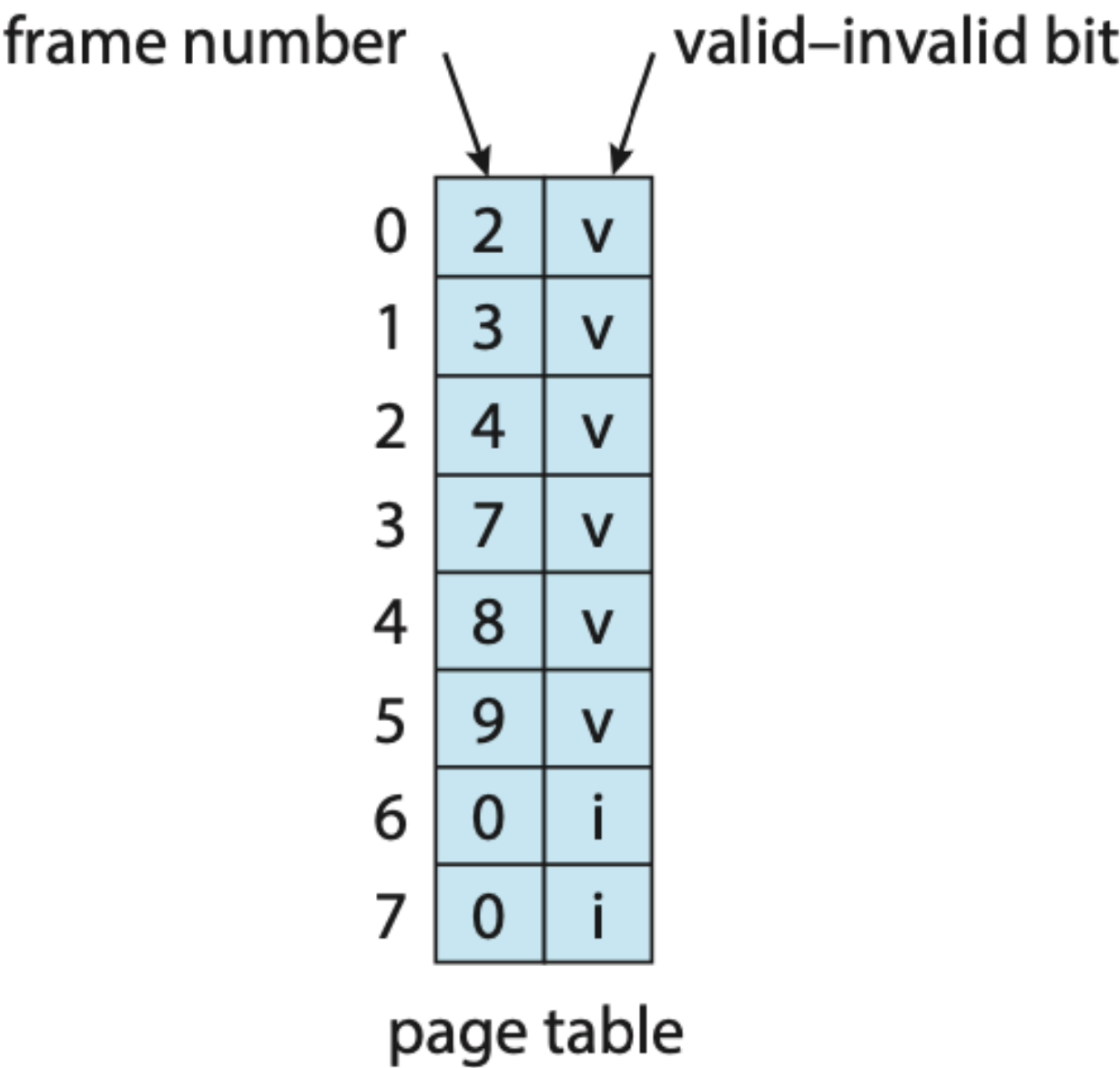
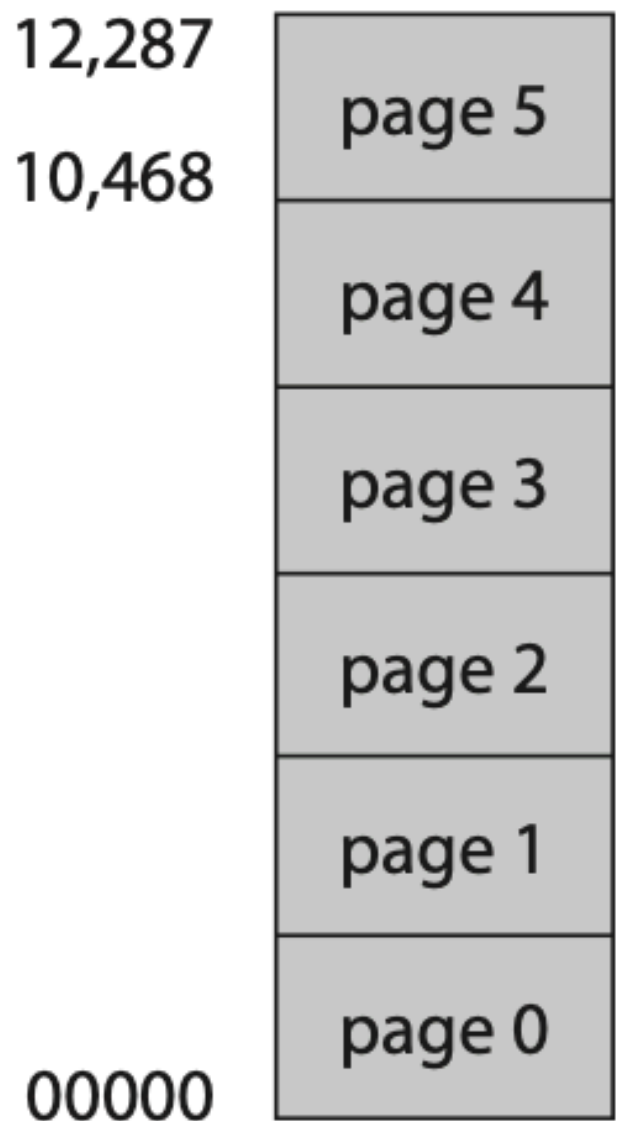
In a paging-based memory management system, the **page table** plays a crucial role in translating virtual addresses into physical addresses and managing memory access. Its key functions include:

🔑 1. Address Translation

- The primary role of the page table is to map **virtual page numbers (VPN)** to **physical frame numbers (PFN)**.
- When a process generates a virtual address, the Memory Management Unit (MMU) uses the page table to translate it into a physical address.

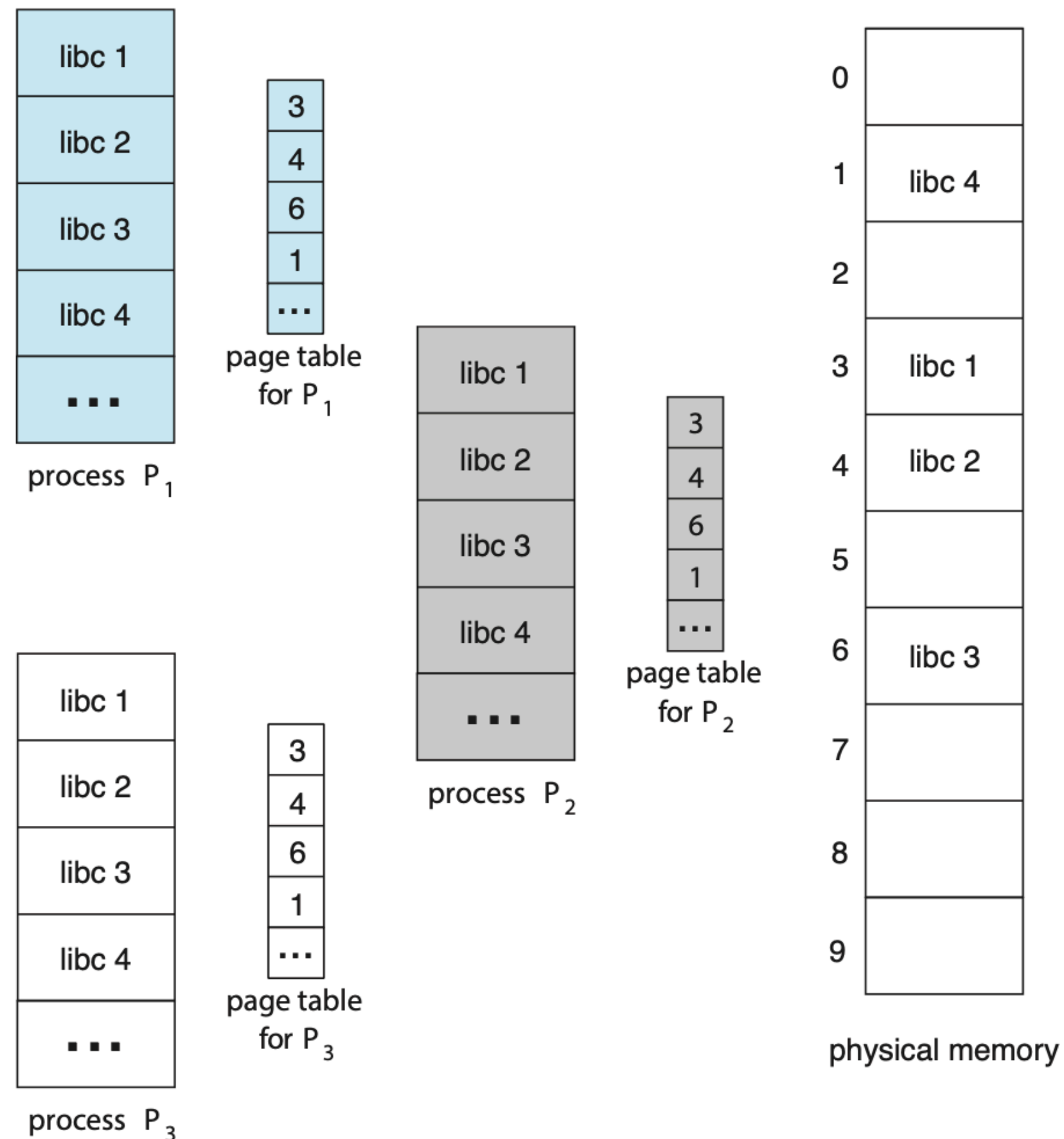
🔒 2. Memory Protection

- Page tables include **protection bits** (e.g., read, write, execute) to control access rights to each page.
- This prevents processes from accessing memory illegally and supports isolation between processes.



Shared Pages

- In computing, a **computer program** or **subroutine** is called **reentrant** if multiple invocations can safely run concurrently. (It can be **shared**.)
- On a typical Linux system, most user processes require the standard C library libc



Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping

Structure of the Page Table

可能会考!

如果一个Process的 Page Table 大小太大，大到一个Frame 放不下時

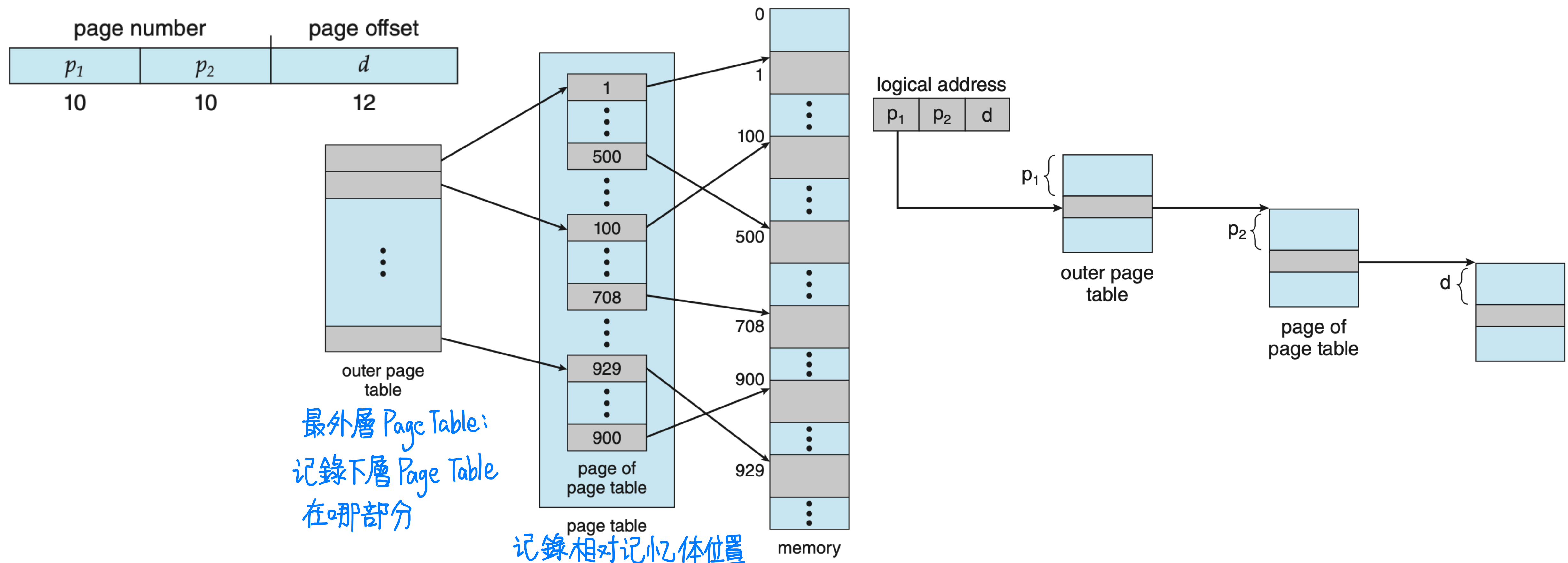
- Hierarchical Paging (linux accepted this)
- Hashed Page Tables
- Inverted Page Tables

Structure of the Page Table

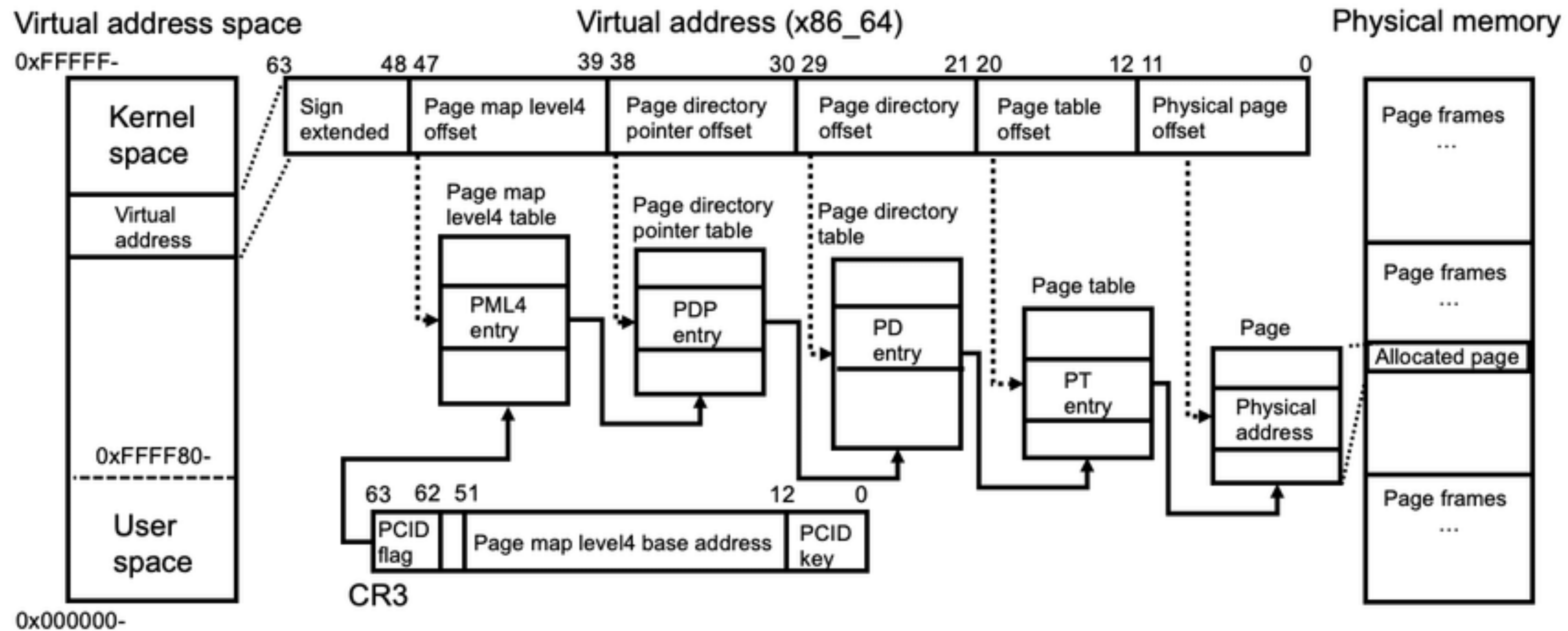
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Paging 階層式 Paging

- Most modern computer systems support a large logical address space.
- The page table itself becomes excessively large such that we would not want to allocate the page table contiguously.



Hierarchical Paging



在linux中總共要讀 5層 Paging (5 is a const, $O(1)$)

Reference: H. Kuzuno and T. Yamauchi, "Mitigation of Kernel Memory Corruption Using Multiple Kernel Memory Mechanism," in *IEEE Access*, vol. 9, pp. 111651-111665, 2021, doi: 10.1109/ACCESS.2021.3101452

<https://ieeexplore.ieee.org/document/9502080>

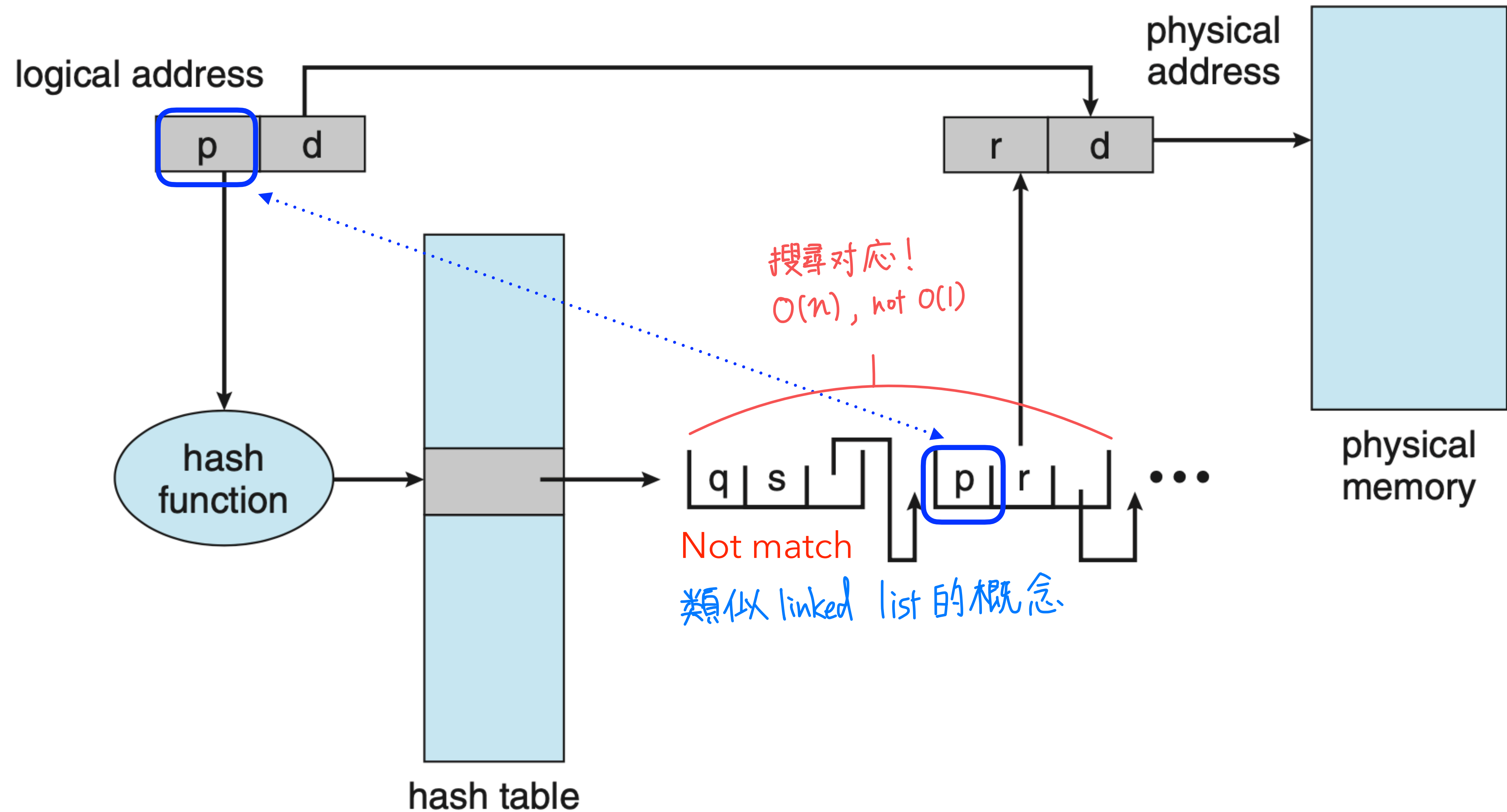
Structure of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hashed Page Tables

- Each entry in the hash table contains a linked list of elements that hash to the same location
- Each element consists of three fields
 - The virtual page number
 - The value of the mapped page frame
 - A pointer to the next element in the linked list

Hashed Page Tables



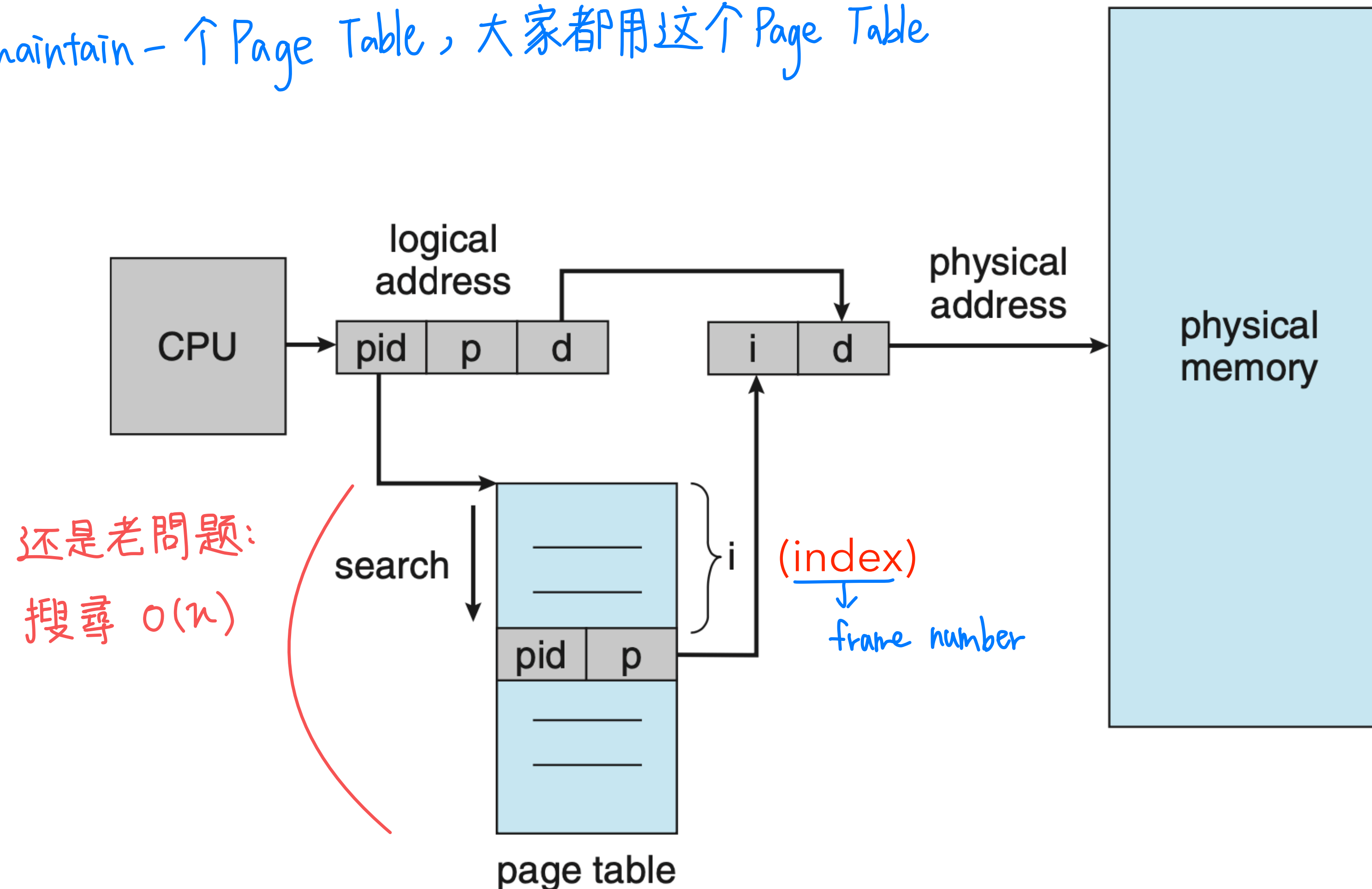
Structure of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Inverted Page Tables

- An inverted page table has one entry for each **frame** of memory. It means that **only one** page table is in the system.

整個系統 maintain 一個 Page Table，大家都用這個 Page Table



Inverted Page Tables

／ 搜尋太花時間了

- It increases the amount of time needed to search the table when a page reference occurs.
- To alleviate this problem, we can use a hash table
／ 為了緩解，可能又要用 hash Table

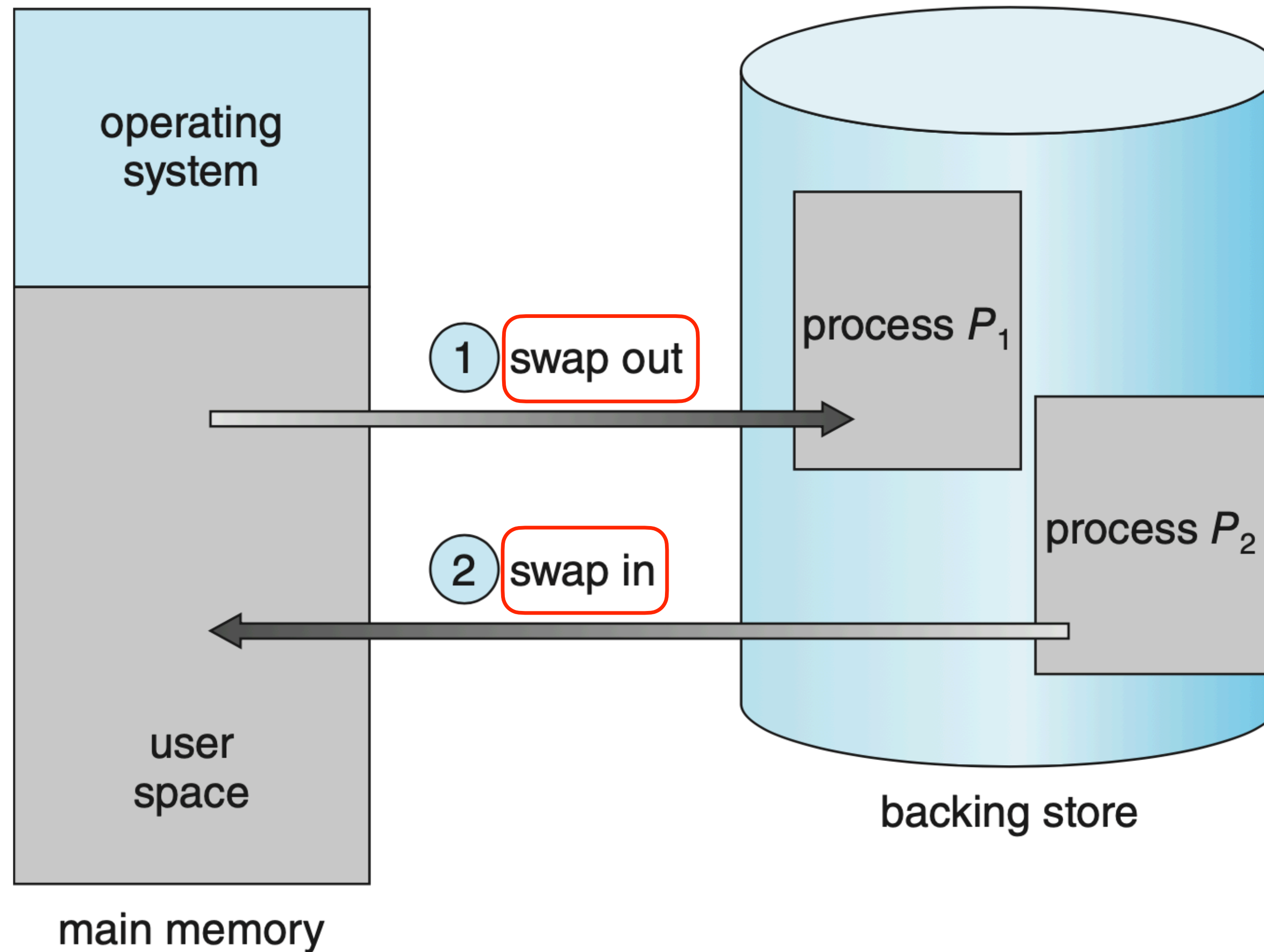
Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping

Swapping

- Process instructions and the data they operate on must be in memory to be executed
- However, a process, or a portion of a process, can be **swapped** temporarily **out** of memory to a **backing store** and then brought back into memory for continued execution
- Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system

Swapping



Swapping with Paging

- Most systems, including Linux and Windows, now use a variation of swapping in which **pages of a process** rather than **an entire process** can be swapped
- In fact, the term swapping now generally refers to stand swapping (an entire process), and paging refers to swapping with paging
 - page out, page in

Swapping with Paging

